

Direct Communication via TCP/IP to the Kistler REST API

Author: Isak Lichtenstein
Date: 23. Apr. 2018
Version: 2.0

1 Table of Contents

2	Introduction	2
3	HTTP Protocol	2
3.1	HTTP-Request.....	3
3.2	HTTP-Response	3
4	Example Explained	4
4.1	Request	4
4.2	Response.....	4
5	Testing	5

2 Introduction

The LabAmp has an Ethernet interface and its communication takes place via HTTP-based requests to the web server. However, this means that all requests must be formatted according to the HTTP protocol.

Many automation systems, e.g. PLCs, have an Ethernet interface which allows communication with other devices via TCP/IP. Since the TCP/IP layer is below the application layer (HTTP) according to the [OSI model](#), no direct data exchange between these layers is possible. Thus, the data must be packed into specific frames containing a header and other information.

The aim of the document is to show by means of an example how to pack the data so that an automation system can communicate directly with a LabAmp from its TCP/IP interface. However, the HTTP protocol must first be briefly explained.

3 HTTP Protocol

The HTTP protocol is publicly available and is well described here:
https://en.wikipedia.org/wiki/Hypertext_Transfer_Protocol

The following explanations are based on the author's subjective understanding of the HTTP protocol. However, this is not complete, which is why minor inaccuracies cannot be ruled out.

This chapter shows but does not explain sample data. A detailed explanation can be found in the following chapter.

Communication via HTTP takes place in the following steps:

1. The client sends a request to the server via TCP/IP (Request)
2. The server processes the request and sends a response to the client (Response)

The HTTP requests/responses each consist of a header and data. The header can contain several fields, each field occupying its own line and being closed by the line break <CR><LF>.

Here is a list of all possible header fields:
https://en.wikipedia.org/wiki/List_of_HTTP_header_fields
The header is separated from the data by a blank line.

3.1 HTTP-Request

According to HTTP V1.1, the following fields are mandatory for a request:

- HTTP request method, internal API address and HTTP version used
- Host: name or address of the server
- Content-Length: Length of the data in bytes

Example:

```
POST /api/$/signal/get HTTP/1.1
Host: 192.168.103.10
Content-Length:32
```

```
{"type":"SENSOR","channels":[1]}
----
```

3.2 HTTP-Response

The answer also consists of several fields, among others:

- HTTP version and response code
- Content-Length or Transfer Encoding

If the server knows the length of the response from the beginning, then it sets the Content-Length field of the header. Otherwise it sends the data with the [Transfer Encoding field](#) set accordingly.

If he sets it to "chunked", each chunk consists of two lines:

- Chunk length
- Chunk data

Example:

```
HTTP/1.1 200 OK
Server: nginx/1.10.1
Date: Mon, 23 Apr 2018 07:45:11 GMT
Content-Type: application/json
Transfer-Encoding: chunked
Connection: keep-alive
X-Powered-By: PHP/5.6.16
Set-Cookie: PHPSESSID=17776d9c3ffb9a5914a25a220b3b898b; path=/
```

```
8f
{"result":0,"data":{"items":[{"ampl":1004313.875,"channel":1,"live":[0]
,"max":1142828.75,"min":[-865798.9375],"rms":[0]}],"type":"SENSOR"}}
0
----
```

4 Example Explained

This chapter explains in detail the examples of the previous chapter.

The aim is to read the current "live" values of input channel 1 of the LabAmp via TCP/IP. According to the documentation, the request must be sent to the API `/api/$signal/get` with the following payload:

```
{"type": "SENSOR", "channels": [1]}
```

4.1 Request

```
Internal API address:    /api/$/signal/get
HTTP method:            POST
Payload:                {"type": "SENSOR", "channels": [1]}
Payload length (String length): 32
IP address (LabAmp IP): 192.168.103.10
HTTP version:           1.1
```

Thus, the HTTP request looks like this:

```
POST /api/$/signal/get HTTP/1.1
Host: 192.168.103.10
Content-Length: 32
```

```
{"type": "SENSOR", "channels": [1]}
----
```

ATTENTION: ALL LINE BREAKS AND BLANK LINES ARE PART OF THE FORMATTING!

As a simple string, the request looks like this:

```
POST /api/$/signal/get HTTP/1.1\r\nHost: 192.168.103.10\r\nContent-
Length: 32\r\n\r\n{"type": "SENSOR", "channels": [1]}
```

4.2 Response

```
HTTP/1.1 200 OK
Server: nginx/1.10.1
Date: Mon, 23 Apr 2018 08:13:11 GMT
Content-Type: application/json
Transfer-Encoding: chunked
Connection: keep-alive
X-Powered-By: PHP/5.6.16
Set-Cookie: PHPSESSID=a768cb81a92c20c6a38c6226cb40f4ec; path=/
```

HTTP version & response code

Server version

Payload format

8f Size of the "Chunks": 8f == 143 Characters

```
{"result":0,"data":{"items":[{"amp1": [1004313.875], "channel":1, "live": [0]
, "max": [1142828.75], "min": [-865798.9375], "rms": [0] }], "type": "SENSOR"}}
```

0 Terminating-Chunk, see [Transfer Encoding Field](#)

Comment

The header formatting and the header content of the response depend on the web server, on which Kistler has little influence. Usually new server versions are backwards compatible. However, this is not explicitly verified during an update.

Ideally, an HTTP library should be used, since the entire HTTP layer is then covered by the library and does not have to be implemented by the developer.

5 Testing

All examples mentioned were tested with the [Packet Sender Tool](#) and LabAmp 5165A4 with firmware version 1.2.5.

To easily view the communication, a REST API request can be sent via [Postman](#) and recorded with [Wireshark](#).